

TinyTorch Debugging & Performance Benchmark Solutions

A practical debugging note for the Linear-layer computational complexity benchmark.

1. Problem: Benchmark Taking Too Long

The benchmark was running for more than 11 minutes. The main reason is that the timing loop executes the TinyTorch Linear layer many times. The original benchmark uses four batch sizes and 100 timing iterations for each batch size, resulting in 400 forward passes, plus warm-up passes.

Original expensive configuration

```
batch_sizes = [1, 32, 128, 512]

# Warm up
for _ in range(10):
    _ = layer.forward(x)

# Benchmark
iterations = 100
for _ in range(iterations):
    _ = layer.forward(x)
```

2. Immediate Debugging Solution

For an educational framework, start with a very small benchmark. Reduce warm-up passes from 10 to 2 and timing iterations from 100 to 5. This makes the experiment fast enough to iterate on while still providing useful measurements.

```
# Warm up
for _ in range(2):
    layer.forward(x)

# Benchmark
iterations = 5

start = time.perf_counter()

for _ in range(iterations):
    layer.forward(x)

elapsed = time.perf_counter() - start
```

Rule

Debug first, benchmark second. Use 3–5 iterations while developing. Increase repetitions only after the benchmark is known to finish quickly and produce sensible results.

3. Why TinyTorch Can Be Slow

Source of overhead	What it means
Python execution	Every operation passes through Python-level code.
Tensor objects	Forward operations may create intermediate Tensor objects.
Autograd	Graph construction and gradient bookkeeping can add overhead.
Shape checks	Validation and bookkeeping happen in addition to numerical work.
Memory allocation	Temporary arrays and tensors may be repeatedly allocated.
Educational design	TinyTorch prioritizes transparency and learning over production speed.

4. MACs vs FLOPs — Important Naming Fix

The original code labels the matrix multiplication count as **matmul_flops**, but the calculation actually represents MACs (multiply-accumulate operations). A MAC consists of one multiplication and one addition. A common approximation is $\text{FLOPs} = 2 \times \text{MACs}$.

```
# Correct terminology
matmul_macs = batch_size * in_features * out_features
bias_macs = batch_size * out_features
total_macs = matmul_macs + bias_macs

# Approximate floating-point operations
total_flops = 2 * total_macs
```

5. Linear Layer Complexity

For a Linear layer with input dimension I, output dimension O, and batch size B:

```
Matrix multiplication MACs =  $B \times I \times O$ 
Bias additions            =  $B \times O$ 
Total MACs                  $\approx B \times I \times O + B \times O$ 
```

Computational complexity = $O(B \times I \times O)$

Example: Linear(784, 256)

Batch	Matrix MACs	Bias MACs	Total MACs
1	200,704	256	200,960
32	6,422,528	8,192	6,430,720
128	25,690,112	32,768	25,722,880
512	102,760,448	131,072	102,891,520

6. Recommended Benchmark Code

```
def analyze_layer_performance():
    """Analyze computational complexity of a Linear layer."""
    import time

    print("Analyzing Layer Computational Complexity...")

    batch_sizes = [1, 32, 128, 512]
    layer = Linear(784, 256)

    print("\nLinear Layer MACs Analysis:")
    print("Batch Size → Matrix Multiply → Bias Add → Total MACs")
    print("Note: FLOPs ≈ 2 × MACs")

    for batch_size in batch_sizes:
        matmul_macs = batch_size * 784 * 256
        bias_macs = batch_size * 256
        total_macs = matmul_macs + bias_macs

        print(
            f"{batch_size:10d} → "
            f"{matmul_macs:15,} → "
            f"{bias_macs:10,} → "
            f"{total_macs:12,}"
        )

    print("\nLinear Layer Timing Analysis:")
    print("Batch Size → Time (ms) → Throughput (samples/sec)")

    for batch_size in batch_sizes:
        x = Tensor(rng.standard_normal((batch_size, 784)))

        # Small warm-up
        for _ in range(2):
            layer.forward(x)

        # Small benchmark during development
```

```

iterations = 5

start = time.perf_counter()

for _ in range(iterations):
    layer.forward(x)

elapsed = time.perf_counter() - start

time_per_forward = (elapsed / iterations) * 1000
throughput = (batch_size * iterations) / elapsed

print(
    f"{batch_size:10d} → "
    f"{time_per_forward:8.3f} ms → "
    f"{throughput:12,.0f} samples/sec"
)

print("\nKey Insights:")
print("Linear complexity:  $O(\text{batch} \times \text{in\_features} \times \text{out\_features})$ ")
print("Larger batches increase total work but may improve throughput.")
print("Measured time includes Python, Tensor, NumPy and autograd overhead.")
print("MAC count and wall-clock runtime are different performance metrics.")

```

7. Debugging Checklist

Step	Action
1	Interrupt a benchmark that has clearly run too long.
2	Check the number of iterations and warm-up passes.
3	Start with 3–5 iterations.
4	Verify the input shape matches the Linear layer.
5	Separate theoretical MAC analysis from timing.
6	Use correct terminology: MACs versus FLOPs.
7	Only increase iterations after the benchmark is stable.

8. Systems-Level Lesson

Concept → **Why** → **Example** → **Rule**

Concept: Computational complexity and measured performance are different things.

Why: MACs describe the amount of mathematical work, while runtime also depends on implementation overhead, memory movement, Python execution, allocations, and framework design.

Example: Linear(784, 256) with batch size 1 has about 200,960 MACs including bias additions, yet a Python/Tensor implementation can still have relatively high wall-clock latency.

Rule: Use theory to explain scaling; use benchmarks to measure implementation behavior. Never assume that fewer theoretical operations automatically means faster real execution.

TinyTorch project note — debugging and performance benchmark reference